SOURCE-GROUNDED TECHNICAL REFERENCE

Factory Chaos Lab: Technical Study Guide

Implementation, execution, debugging, safety, and future design

11 October 2026

Architecture · Execution · Debugging · Safety · Future Design

Lab revision 6527f296 · FactorySemantics pin b90ca1c8

Full study guide, glossary and 42 technical interview questions

Contents

How to use this guide **4**

1. Problem statement, objectives, constraints, evaluation, and Day 2 **4**
1.1 Available problem framing . . . 4
1.2 Why the problem matters . . . 4
1.3 Implemented objectives and constraints . . . 5
1.4 Evaluation that can be justified from the project . . . 5

2. Project overview and end-to-end architecture **5**
2.1 Three implemented domains and their boundaries . . . 5
2.2 Two experiment layers . . . 6
2.3 MES integration and data flow . . . 6

3. Repository structure and important file responsibilities **7**
3.1 Root, configuration, definitions, and documentation . . . 7
3.2 Core modules . . . 7
3.3 Scripts and monitoring artifacts . . . 8
3.4 Backend and retained generated state . . . 9

4. Technology stack and actual usage **9**

5. Major subsystems: purpose, implementation, interfaces, interactions **10**
5.1 Orchestration and controller . . . 10
5.2 Policy and resource enforcement . . . 10
5.3 Fault registry and injection . . . 11
5.4 MCP and local API . . . 11
5.5 Agents and hosted model transport . . . 11
5.6 Recovery, watchdog and emergency stop . . . 12
5.7 Monitoring, dashboard, and reports . . . 12
5.8 Persistence and audit . . . 12
5.9 Generic RAG backend . . . 13
5.10 Process names and SQL hook integration . . . 13

6. Classes, APIs, schemas, configuration, and dependencies **14**
6.1 Experiment contract and meaningful fields . . . 14
6.2 States versus results . . . 14
6.3 Data contracts to recognize . . . 14
6.4 Configuration and active dependencies . . . 15
6.5 HTTP surfaces and ports . . . 15

7. End-to-end execution walkthroughs **16**
7.1 Fresh setup and startup . . . 16
7.2 No-fault infrastructure baseline . . . 16
7.3 API crash through direct CLI . . . 16
7.4 Same experiment through the Injector . . . 16
7.5 Analyst explanation after a run . . . 17
7.6 Application breakdown and scoring . . . 17
7.7 Backend upload, retrieval and answer . . . 17
7.8 Evidence retrieval and comparison . . . 17

8. Fault mechanisms, approval, limits, cleanup, and recovery **18**
8.1 Infrastructure mechanisms . . . 18
8.2 Simulator mechanisms . . . 18
8.3 Approval and enforcement boundaries . . . 18
8.4 Cleanup is a journal-based protocol with limits . . . 19
8.5 Sequential versus simultaneous injection . . . 19

9. Observability and how recovery is measured **19**
9.1 Sampling and metrics . . . 19
9.2 Five-stage infrastructure recovery . . . 19

9.3 Application recovery evidence . . . 20
9.4 Exported series, event flow and dashboards . . . 20
9.5 Measurement caveats to explain . . . 20

10. Docker, deployment, networking, and host/container boundaries 21
10.1 What Docker does and what exists here . . . 21
10.2 Boundary differences and safe implications . . . 21
10.3 Independent backend deployment . . . 22

11. Testing, debugging, failure scenarios, security, and trade-offs 22
11.1 Tests and verified scope . . . 22
11.2 Debug by lifecycle stage . . . 23
11.3 Confirmed source risks versus demonstrated failures . . . 24
11.4 Security model and limitations . . . 24
11.5 Design choices to justify accurately . . . 24

12. Limitations and unfinished deliverables 25
12.1 Requirement and evidence gaps . . . 25
12.2 Capability gaps . . . 25
12.3 Safety and measurement gaps . . . 25

13. Future implementation plan 25
13.1 Dependency-ordered foundation . . . 25
13.2 Recommendation engine . . . 25
13.3 API-crash injection . . . 27
13.4 Multiple injections . . . 28
13.5 Broader delivery sequence . . . 29

14. Technical interview preparation and glossary 29
14.1 Beginner questions with grounded answers . . . 29
14.2 Intermediate questions . . . 29
14.3 Advanced questions and failure cases . . . 31
14.4 Glossary . . . 32
14.5 Final cross-check and unresolved gaps . . . 33

How to use this guide

This guide explains the lab at commit `6527f29645bd2f1990cb9b8c34338e034e4c2184` and its pinned FactorySemantics MES dependency at `b90ca1c81e3b520be629999fff8bd9ffb82ad514`. Paths are relative to the repository root. A symbol following a path identifies the implementation to consult; line numbers are deliberately omitted because they change. PROJECT_AUDIT.md records evidence and gaps; EXECUTION_FLOWS.md provides the execution index.

Use these evidence labels throughout:

- **Implemented:** present in the inspected source; not necessarily exercised end to end here.
- **Verified here:** an explicitly reported command or functional check executed during this review.
- **Historical documentation:** a claim in existing docs, not a new verification.
- **Proposed:** a suggested design or next step; not an agreed team requirement.
- **Unavailable:** the source or evidence needed to make the claim was not found.

Read §§1–4 for context, §§5–9 for code and execution, §§10–12 for operational boundaries, §13 for future development, and §14 to practice explanations. References to an upstream file require the dependency checkout produced by setup. No application code was changed to create this guide.

1. Problem statement, objectives, constraints, evaluation, and Day 2

1.1 Available problem framing

The exact repository terminology is **“AI-Driven Chaos Engineering for Industrial Resilience.”** README describes a **“local, bounded chaos-engineering laboratory around FactorySemantics MES.”** It runs a simulated bottling plant, injects controlled faults, measures consequences, verifies recovery from application evidence, and records JSON results. These statements establish the implemented project’s purpose; they are not a substitute for an original Bosch problem statement.

The original Bosch Hackathon problem statement and Day 2 deliverable document are **unavailable** in this checkout and its accessible history. Searches covered tracked files, accessible project directories, local branches/references, all 11 reachable commits, historical filenames, and relevant historical text. The only Bosch-specific source evidence is the backend title/service naming. No official evaluation weights, mandatory deployment topology, team role list, schedule, or Day 2 acceptance checklist was found. Do not quote invented deliverables in a presentation.

The audit and execution documents were created during this task from code, rather than recovered from earlier files. If official requirements arrive later, add a verbatim requirement-to-evidence matrix and identify conflicts with this guide. Until then, recommendations, API-crash extension work, and multiple-injection designs in §13 are proposals or user-requested study topics.

1.2 Why the problem matters

A manufacturing execution system (MES) interprets machine observations into work-order progress, quality, downtime, and production KPIs. A web endpoint can return 200 while machine ingestion has stopped. Conversely, a production line can continue while its observer is disconnected, leaving uncertain or incomplete accounting. A useful resilience experiment therefore asks more than whether a PID exists: can the MES observe the line, answer an authenticated operation, continue ingesting, and report evidence honestly after a fault?

Chaos engineering means testing a specific hypothesis under a controlled disturbance. In this project the **blast radius** is a single allowlisted simulated plant on one host. A baseline establishes normal behavior; a bounded fault perturbs one resource; cleanup removes the disturbance; recovery checks verify application behavior. It is a laboratory, not permission to experiment on a real factory.

1.3 Implemented objectives and constraints

Objective or constraint	Implementation evidence	What it does not establish
Reproducible target	`UPSTREAM_COMMIT`, `scripts/setup.sh`, copied pack, seeded replay	All transitive lab dependencies are not frozen by a lab lockfile
Deterministic safety	`PolicyValidator`, allowlist, duration/resource bounds	Complete protection against every crash/concurrent-submission interleaving
Real software disturbances	Process signals, SQLite lock, SQL delay, CPU pressure, scoped caps	Network partitions, PostgreSQL outages or container-wide faults
Manufacturing scenarios	Simulator events and upstream scoring harness	Recovery of persistent infrastructure
Agent tool use	Injector/Analyst, MCP schemas, approval callback	An autonomous remediation/recommendation engine
Measured recovery	Staged verifier, samples, integrity, watched states	No lost transaction or semantic correctness of every record
Explainable record	JSON result, audit trail, logs, summaries and comparison	Production-grade audit durability or tamper resistance
Modest local resources	Python 3.12+, SQLite, no required Docker/GPU	Equal compatibility on restricted containers or non-Linux hosts

README specifies Ubuntu 24.04, Python 3.12+, Git/venv, and roughly 1.5 GB available beyond other activity. The audited host has Python 3.12.14 and enough memory, but no user systemd manager. The main lab uses host memory sampling; the backend has a separate cgroup-aware memory implementation.

1.4 Evaluation that can be justified from the project

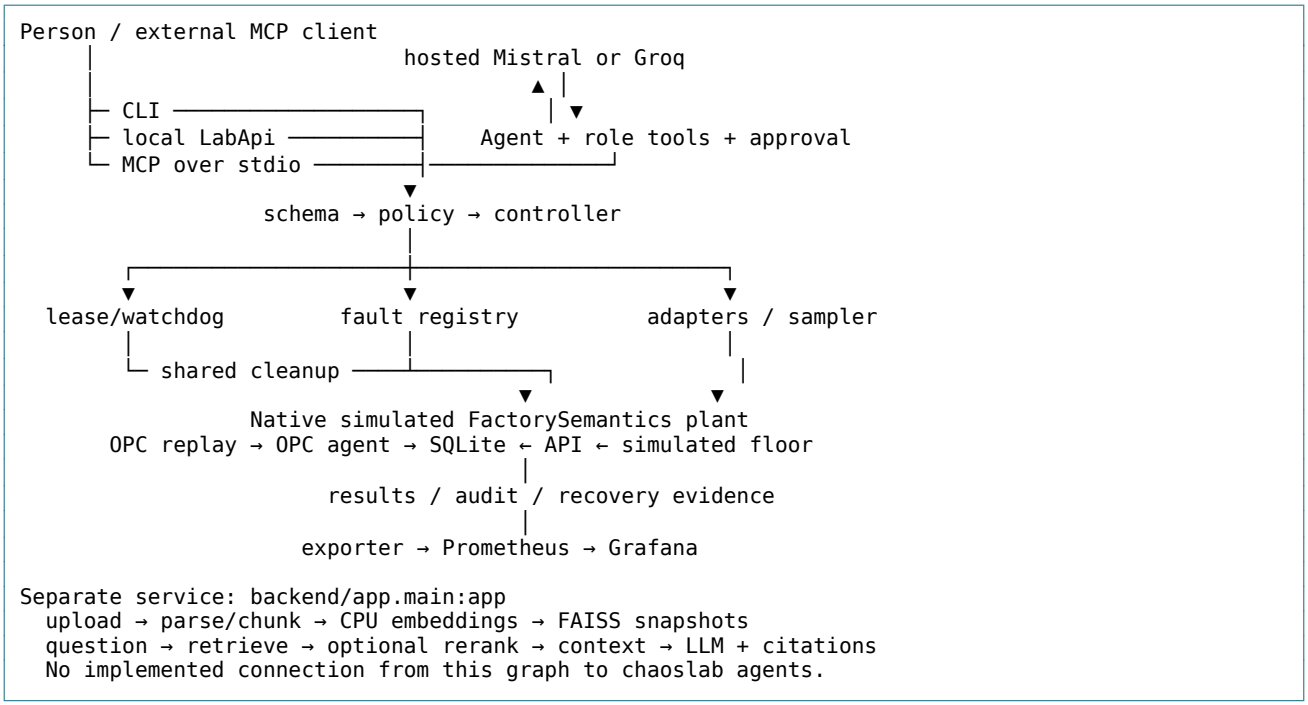
Without official criteria, use **project-derived validation questions**, explicitly labelled as such: Was the proposal rejected or accepted for an understandable reason? Did the fault actually happen? Was its duration bounded? Was cleanup independently verified? Did the affected software and manufacturing observations return? Are missing measurements preserved as unavailable? Can a person inspect the record and reproduce the definition? Does agent analysis cite observed evidence?

Do not collapse these questions into “the experiment passed.” Rejection, injection failure, failed cleanup, service recovery with data findings, and full recovery are different outcomes. §11 explains the actual tests; §12 lists unverified or unfinished work.

2. Project overview and end-to-end architecture

2.1 Three implemented domains and their boundaries

The repository contains a deterministic chaos lab, LLM agents driving that lab, and a separate generic document-RAG backend. The target MES is a pinned upstream dependency. These pieces are not one fully connected AI product: the agents call lab tools; the RAG backend handles uploaded documents; there is no source path joining RAG answers to fault selection or MES remediation.



The controller does not depend on hosted AI or monitoring servers. Prometheus and Grafana are optional presentation/storage layers over measurements. Agents add reasoning and tool sequencing above deterministic enforcement.

2.2 Two experiment layers

Infrastructure experiments change whether running software works: kill/freeze a component, hold a SQLite write lock, delay SQL statements, contend for CPU or impose resource caps. They require the persistent plant healthy before starting. They use live probes, process observations and staged recovery.

Application experiments change what a simulated manufacturing line does: breakdown, changeover, micro-stops, counter reset, scrap burst, starve, block or OPC disconnect. They generate a scenario plan and call an upstream harness in an ephemeral plant. Ground-truth simulator behavior is compared with MES accounting. They do not kill the persistent plant and do not provide its infrastructure recovery time.

This boundary prevents a false claim: correctly detecting a scripted machine breakdown does not prove that an API survives a crash. Likewise restarting an API does not prove its manufacturing accounting matches truth.

2.3 MES integration and data flow

`factorysemantics-mes/src/fsmes/plant.py::plant_env/start` supplies the target process configuration. The native lab contains `run-opc-sim--replay`, `run-opc-agent`, `run-api`, `run-operations`, plus a log sink. OPC UA is the machine telemetry protocol; a replay server publishes recorded signals, and the OPC agent subscribes and books observations/production into SQLite. The floor simulator performs operator-like activity against the API. The API reads manufacturing state and exposes health/metrics/operator screens.

Lab integration is concentrated in `chaoslab/adapters/factorysemantics.py::FactorySemanticsAdapter`: HTTP routes, authentication, discovery, environment, signal and restart commands. Other modules operate on normalized samples and symbolic components. Changing targets should involve a new adapter and target contracts, not scattered endpoint strings throughout the controller.

3. Repository structure and important file responsibilities

3.1 Root, configuration, definitions, and documentation

Path	Responsibility and study priority
`README.md`	Commands, fault inventory, agents, monitoring, tests; some older claims are stale
`UPSTREAM_COMMIT`	Exact target revision; preserve it when comparing results
`requirements-lab.txt`	Pinned direct lab requirements; dependency target is installed separately
`.env.example`	Agent environment example; does not supply usable credentials
`.mcp.json`	Registers `.venv/bin/python -m chaoslab.mcp` for a compatible external client
`.gitignore`	Excludes virtualenv, upstream dependency, runtime, results, local environment and caches
`config/lab.yaml`	Paths, plant timing/speed, reader settings, sampling, harness timeout, exporter
`config/safety-policy.yaml`	Bounds and enablement switches; authoritative for deterministic acceptance
`config/service-allowlist.yaml`	Target components/commands, fault-target mappings, database clients
`experiments/definitions/*.yaml`	Seventeen stored experiment definitions; fault vocabulary is larger than stored inventory
`docs/ARCHITECTURE.md`	Earlier A/B/C/D design; AI absence claims require correction by current code evidence
`docs/TARGET_APPLICATION.md`	MES interfaces and historical observations; useful upstream navigation
`docs/SAFETY.md`	Intended guarantees and limitations; audit adds implementation caveats
`docs/DATABASE_LATENCY.md`, `docs/RESOURCE_LIMIT.md`	Fault-specific mechanism and evidence explanation
`docs/TROUBLESHOOTING.md`, `docs/LICENSES.md`	Operational symptoms and dependency licensing
`docs/PROJECT_AUDIT.md`, `docs/EXECUTION_FLOWS.md`	Current evidence index and traced paths created by this task

`docs/EXPERIMENTS.md` and `docs/RESOURCE_ASSESSMENT.md` are referenced but missing. `results/.gitkeep` is the only tracked result placeholder. Generated runs are evidence in the current machine, not committed historical reports.

3.2 Core modules

Path / symbols	Responsibilities
`chaoslab/__main__.py`, `chaoslab/__init__.py`, `chaoslab/cli.py::main/cmd_*`	Module entry, version, argparse dispatch, JSON/human output, exit codes
`chaoslab/settings.py::Settings/load`	YAML loading, required-key checks, derived paths, fleet/pack settings and executable resolution
`chaoslab/model.py::Experiment/Step/State/from_mapping`	Definition schema and state-transition vocabulary
`chaoslab/policy.py::Decision/PolicyValidator`	Semantic acceptance, normalization and estimates
`chaoslab/resources.py::HostSnapshot/ResourceGate`	Host resource checks and reports
`chaoslab/lifecycle.py::Lifecycle`	Prepare/start/stop/status of plant and exporter
`chaoslab/api.py::LabApi`	Shared Python discovery/run/submit/read/stop contract
`chaoslab/mcp_server.py::build_server`	21 thin MCP wrappers and tool annotations
`chaoslab/chaos/controller.py::Controller/StateTracker`	Experiment orchestration and verdict assembly
`chaoslab/chaos/faults.py::Fault/FAULTS`	Fault specs, parameter bounds, inject/cleanup/removal implementations

Path / symbols	Responsibilities
chaoslab/chaos/cleanup.py::cleanup_records	Shared reversed-record cleanup for controller/watchdog/emergency stop
chaoslab/chaos/recovery.py::RecoveryVerifier	Stable service/operation/manufacturing/data evidence
chaoslab/chaos/watchdog.py::spawn/run	Separate lease-bound cleanup process
chaoslab/chaos/holders.py	Bounded CPU, SQLite lock and resource-reset helpers
chaoslab/chaos/cgroups.py	Derive safe scope names; write/read/lift kernel limits and counters
chaoslab/audit.py::AuditLog/Lease	Append audit events; active lease/stop flag and atomic state replacement
chaoslab/store.py::ResultStore/flatten/compare	Run IDs, result storage, CSV, result comparison
chaoslab/report.py	Parameters, result, phase metrics and fault-specific human report
chaoslab/proctitle.py::set_name/set_title	Short process names versus full titles; naming must preserve discovery
chaoslab/adapters/factorysemantics.py	Only target-aware HTTP/process/restart integration
chaoslab/adapters/metrics.py::MetricsAdapter/Sampler/HostSampler	Live samples, distributions, DB/cgroup evidence and phases
chaoslab/adapters/scoring.py::ScoringAdapter	Generated TOML/harness supervision/parsing/line-state timeline
chaoslab/adapters/logs.py::LogCollector/redact	UTC-window log excerpts and best-effort redaction
chaoslab/plant_names/sitecustomize.py, chaoslab/plant_names/chaoslab_dbhook.py::Hook	Names and startup-activated SQL listeners in target processes
chaoslab/workload/generator.py::WorkloadGenerator, chaoslab/workload/profiles/*.yaml	Bounded read traffic and profile discovery
chaoslab/observability/exporter.py::Collector/serve	Cached loopback Prometheus exposition
chaoslab/agents/roles.py, chaoslab/agents/runtime.py, chaoslab/agents/commands.py, chaoslab/agents/llm.py	Role instructions, dispatch/approval, orchestration, hosted chat transport

Package chaoslab/__init__.py files mainly establish importable modules; they do not hide additional service orchestrators.

3.3 Scripts and monitoring artifacts

scripts/common.sh centralizes root/virtualenv and chaoslab invocation. setup.sh, start.sh, stop.sh, status.sh, health-check.sh wrap lifecycle commands. run-experiment.sh wraps a definition; baseline.sh is a convenience baseline entry. emergency-stop.sh invokes shared emergency logic. collect-logs.sh and export-results.sh expose evidence collection/export. watch.sh uses the shipped observability/htop/htoprc.

inject-db-latency.sh and inject-resource-limit.sh validate CLI parameters, write local definitions, and use inject-common.sh for direct or agent execution and analyst reporting. Inspect their shell exit behavior separately from controller verdicts: a wrapper printing a report is not proof the run recovered. inject-opc.sh depends on a missing separate injections/chaos-opcua project; use the built-in OPC process-outage definition for the implemented lab path.

install-monitoring.sh, start-monitoring.sh, stop-monitoring.sh manage optional downloaded monitoring binaries and local process/PID state. observability/prometheus/prometheus.yml selects scrape targets; rules/lab.rules.yml defines rate recording rules and informational alerts. observability/grafana/build_dashboards.py::Board generates three dashboard JSON files and their layout; provisioning YAML supplies the Prometheus data source and dashboard directory.

3.4 Backend and retained generated state

Backend file map: `backend/app/main.py` is the FastAPI entry; `middleware.py` handles timing/memory shedding; `os_limits.py` sets threads/affinity/niceness and reads cgroups. `app/rag/config.py`, `schemas.py`, `errors.py` define settings, contracts and errors. `api/` exposes documents, retrieval, query and health. `ingestion/` parses/chunks/deduplicates; `embeddings/` supplies lazy CPU embeddings; `vectorstore/faiss_store.py` persists vectors and metadata; `retrieval/` supplies top-k and optional reranking; `generation/` constructs prompts/providers/citation checks; `pipeline/rag_pipeline.py` joins these using LangGraph; `evaluation/metrics.py` computes retrieval metrics; `rebuild.py` stages offline reindexing. `backend/tests`, requirements, `.env.example`, `pytest.ini`, `deploy.sh`, and resource scripts document/test that independent workflow.

Generated lab state lives under `runtime/fleet`, `data`, `work`, `state`, and `logs`; results are under `results/<id>`. `.venv` holds dependencies; `factorysemantics-mes/` is an ignored dependency checkout. Do not classify its Docker files or tests as top-level lab implementation. Resetting runtime deletes plant state; it does not preserve runs by magic—results must remain separately retained.

4. Technology stack and actual usage

Technology	Background	Actual project use
Python 3.12+	Runtime and standard-library process/network/thread tools	Dataclasses, argparse, subprocess, urllib, monotonic timing, TOML reading
PyYAML	YAML parser	Definitions, policy, allowlist and workload profiles; `safe_load`
psutil	Process/system metrics and control	Discovery, RSS/status, signals/affinity, memory and CPU sampling
pytest	Automated test runner	Lab unit/component tests and backend deterministic tests
MCP SDK 2.2.0	Structured tools for model clients	Local stdio server plus in-process agent tool calls; not an HTTP gateway
setproctitle 1.3.7	Rewrites process title	Runner/agent/watchdog titles only; plant discovery-sensitive processes use short comm names
SQLite	Embedded transactional database with one writer	Persistent plant per-plant file, safe write-lock fault, quick integrity check
SQLAlchemy / Alembic	Database abstraction / migrations	Upstream MES database; lab SQLAlchemy listeners inject latency
asyncua / OPC UA	Machine telemetry protocol implementation	Upstream replay server and subscriber agent
FastAPI / Uvicorn	HTTP API framework / ASGI server	Upstream MES and independent backend; lab itself exposes only exporter HTTP
httpx	Async HTTP client	Agent hosted chat completions; backend provider/test transport
Linux signals	OS process state control	KILL/TERM outage, STOP/CONT freeze
cgroup v2 / systemd scopes	Kernel resource grouping/delegation	Per-component RAM/CPU caps, reset helper, counters; requires host support
Prometheus / Grafana	Time-series collection / visualization	Optional pulls of exporter/MES metrics and provisioned dashboards
Mistral / Groq	Hosted OpenAI-compatible chat services	Agent LLM proposals and evidence interpretation; credentials/network required
LangChain / LangGraph	Component interfaces / state-graph composition	Backend documents/embeddings/retriever/chat models and RAG execution graph
Sentence Transformers / CPU PyTorch	Text-to-vector model runtime	Lazy local semantic embeddings; no GPU requirement
FAISS IndexFlatIP / NumPy	Exact inner-product search / arrays	Normalized vectors, cosine similarity, local persisted index
PyMuPDF / python-docx / CSV	Content extraction	RAG parsers, no OCR implementation

Technology	Background	Actual project use
filelock	Cross-process filesystem lock	RAG store reload/write serialization; not used by the lab Lease
Gunicorn / systemd	Multiprocess server / service manager	Backend deployment only; four-worker default, one suggested for low RAM
Docker / PostgreSQL / nginx	Containers / server DB / reverse proxy	Alternative upstream Compose deployment, not used by native lab
ONNX Runtime	Inference engine	Installed backend dependency; `/predict` does not load an ONNX model

Direct lab pins are `pyyaml==6.0.3`, `psutil==7.2.2`, `pytest==9.1.1`, `mcp==2.2.0`, `setproctitle==1.3.7`. Backend pins differ, including psutil 6.1.1 and pytest 8.3.4, and have heavyweight numerical dependencies. Use separate virtualenvs. Do not force backend versions into the working lab virtualenv.

5. Major subsystems: purpose, implementation, interfaces, interactions

5.1 Orchestration and controller

`Controller` is the deterministic run coordinator. Its constructor builds policy-facing settings, audit/lease/store, MES adapter, metrics/scoring/log adapters, and resource gate. `StateTracker.to` checks `TRANSITIONS`, records history/audit, updates lease state, and updates the visible runner title. The state machine makes lifecycle order explicit; a completed state does not mean a recovered verdict.

`run` creates the result, validates policy, gates host resources, handles dry run, acquires lease, spawns watchdog and selects the category. Exceptions carry a bounded traceback, attempt recorded cleanup and produce `error` or `unsafe_cleanup_failed`. `finally` releases the lease. Category-specific methods handle baseline/injection/scoring/recovery. Result publication and audit are centralized in `_finish`.

Pseudocode of the principal boundary:

```
experiment = parse_definition(raw)
decision = policy.validate(experiment)
if not decision.allowed: persist_rejection()
elif not resource_gate.allowed: persist_precondition_rejection()
elif dry_run: persist_dry_run_ok()  # no live fault checks
elif lease.acquire():
    spawn_watchdog()
    try:
        run_infrastructure() if infrastructure else run_application()
    except Exception:
        cleanup_recorded_effects_and_record_error()
    finally:
        release_lease()
```

The controller is orchestration, not an LLM agent. A model cannot bypass validation by describing a fault convincingly. However, the lease implementation has concurrency limitations (§11); deterministic logic is not automatically race-free logic.

5.2 Policy and resource enforcement

Schema answers “does this look like a definition?” Policy answers “is this operation allowed?” Fault preconditions answer “can the current plant perform it safely?” These are separate layers.

`PolicyValidator.validate` checks compound enablement/step count, baseline limits, workload profile/rate, each fault layer/parameters/target, application events/speed/duration, estimated wall time, and recovery timeout. `_validate_target` rejects caller-supplied host/address/IP/URL/port, enforces symbolic plant/component/database, and checks simulator station membership. `valida`

te_params rejects unknown parameters, resolves defaults, enforces types/choices/ranges. Decision.normalised becomes controller input.

ResourceGate.check samples available memory and CPU before startup/runs. _run_infrastructure also checks plant RSS. These are initial gates; they do not continuously cancel a run when host memory falls later. Scope resource caps are separate fault mechanisms, not the gate itself.

5.3 Fault registry and injection

FaultSpec describes name, layer, parameters, target keys, effects, cleanup, criteria, prerequisites, support status. Param describes individual bounds/defaults. FaultContext supplies settings and adapter. Check serializes success and detail. Fault defines preconditions, optional prepare, intent, inject, cleanup, removed, and hold_seconds. Registry discovery and execution share the same objects.

The interface separates removal from recovery: a burner can be stopped while the API is still unhealthy. Every added fault must implement both cleanup and independent removal evidence, and must allow a crash-cleanup actor to use a persisted record. §8 compares the actual mechanisms.

5.4 MCP and local API

MCP (Model Context Protocol) lets a client discover tool schemas and call named tools with structured arguments. Here build_server exposes **21** tools over stdio; .mcp.json starts it in the project directory. Agents also call that same server in-process through ToolBox, without a separate stdio client connection.

Group	Tool names
Discovery	list_fault_types, list_targets, get_safety_policy, list_experiments, get_experiment
Execution	validate_experiment, run_experiment, experiment_status, wait_for_experiment, emergency_stop
Evidence	list_runs, get_result, get_metrics, get_logs, get_scoring, get_recovery, compare_runs
Plant	plant_status, check_resources, plant_start, plant_stop

run_experiment, emergency_stop, plant_start, plant_stop have destructive annotations; other tools have read hints. Hints are not a security gate. The actual approval callback exists in agent dispatch only. Dry-run execution still writes a result/audit record, so destructive classification is not based solely on whether a fault occurs.

LabApi is the Python contract: discovery, definitions, validate, blocking run, detached submit, status, evidence accessors, compare, and emergency stop. submit acceptance does not guarantee preflight or recovery: the child revalidates and may refuse. A wait timeout reports unfinished status and leaves the experiment running. Expected errors become ToolError; the wrappers do not swallow every unexpected crash as a successful result.

5.5 Agents and hosted model transport

Role stores name, prompt, allowed tools, and max_rounds=16. INJECTOR receives discovery/status/validate/run/wait/emergency tools, but no plant start/stop or policy-change tool. ANALYST receives read-only result/compare/plant status tools. Agent.run sends system/task messages and function schemas, dispatches tool calls, feeds tool results back, and stops when the model answers without tools or reaches the round cap.

Agent._dispatch parses JSON arguments as an object, checks role membership, gates real run_experiment, calls tool, and records transcript. ToolBox.call serializes structured output or text and truncates at 12,000 characters. The same model can generate multiple tool calls in a round. Procedure and one-experiment discipline live in prompts; runtime does not count accepted real submissions or enforce a denied-call latch.

LLMConfig.from_env selects mistral by default, MISTRAL_API_KEY, base https://api.mistral.ai/v1, model ministral-14b-latest; Groq uses GROQ_API_KEY and requires a named model.

Base/model override variables are supported. `ChatClient.chat` posts OpenAI-shaped messages/tool schemas to `/chat/completions`, retries transport and 429/500/502/503/504 up to four retries, uses exponential waits or numeric Retry-After capped at 60 seconds, and parses `Reply/ToolCall`. Free-tier availability in comments is historical, not a guaranteed provider contract.

`commands.inject` derives the emergency-stop flag from actual transcript calls, maps verdict to CLI exit code, and saves transcript. `commands.analyze` resolves a finished run, checks claimed baseline comparison against actual compare arguments, and writes `agent-analysis-*.json` beside the controller record. `extract_json` accepts a dictionary in text/fence/braces; it does not validate all verdict fields. Analyst explanation is supplementary evidence, not authority to overwrite controller truth.

5.6 Recovery, watchdog and emergency stop

Recovery is a conjunction of separate observations, explained precisely in §9. `RecoveryVerifier` consumes sampler snapshots and a verified fault-removal check. It waits for a stable sequence rather than trusting one successful request.

`watchdog.spawn` starts a detached helper with experiment ID. `watchdog.run` exits for a missing/released/different lease; otherwise it cleans records if the controller disappears or deadline passes. It shares `cleanup_records` with controller/error paths and emergency stop. This reduces divergence between cleanup implementations. It remains a process on the same host, cannot survive host failure, and does not terminate an overdue controller.

`LabApi.emergency_stop` writes a stop flag; controller sleeps check at 250 ms intervals. The API waits up to 20 seconds, forces recorded cleanup if necessary, then sweeps stopped components/helpers/activation/scopes. Sweep is a recovery aid, not a full undo journal. A missing component without an active outage record is not automatically restarted by the sweep.

5.7 Monitoring, dashboard, and reports

`Collector` exposes loopback `/metrics` and `/healthz` through `ThreadingHTTPServer`, caching rendering for one second. Healthz proves exporter responsiveness, not plant health. Metrics encode API/component health, machine counts, production, states, memory, active experiment and latest result. Prometheus pulls them; Grafana queries Prometheus; no browser dashboard controls a fault.

Three Grafana boards: **Lab overview**, **Manufacturing resilience**, and **Experiment comparison**. `Board` assigns IDs/grid placement and common data source; target expressions reference series. Machine state codes map stopped=0, running=1, starved=2, blocked=3, down=4, changeover=5. Result comparison dashboard reflects exported latest-result/time-series information, not arbitrary full JSON score ingestion. Harness scoring remains accessible through run tools/report files.

The MES also has its own operator dashboard/screens in `factorysemantics-mes/src/fsmes/api/routers/dashboard.py` and line view in `api/routers/line.py` under the same upstream source root. That is distinct from Grafana and the independent backend API. There is no separate custom lab web dashboard/frontend in the tracked project.

`chaoslab/report.py`, `ResultStore.flatten`, and `compare` make per-run evidence usable without monitoring servers. Deltas should be compared within the same layer/target/workload/seed/time assumptions. A baseline of another speed or deployment is not automatically meaningful.

5.8 Persistence and audit

`new_run_id` combines UTC timestamp, experiment name and random six-hex suffix. `RUN_ID_PATTERN` constrains directory access; `resolve` accepts exact/unambiguous fragments and special latest/last. `write_result` replaces a temporary JSON file; run creation refuses an existing directory. Only directories with a result file are listed as completed runs.

Result stores definition/version/deployment/commit, state history, resources, initial/final health, injected step records, workload, phase metrics, scoring, cleanup, recovery, reasons and limitations. Samples and excerpts accompany it. CSV emits `unavailable` for missing values.

`AuditLog.record` appends UTC JSON lines with PID and experiment ID. A process-local threading lock protects writes within that process. `Lease._write` uses `os.replace`; this prevents torn JSON visibility but not multi-process acquisition races or durable transactional audit. `active.json` is coordination state, not the completed-result database.

5.9 Generic RAG backend

RAG (retrieval-augmented generation) retrieves document passages and places them in the prompt before asking a model for an answer. It is different from fine-tuning and from an action recommendation engine. This backend has no built-in factory knowledge.

`app.main:app` applies OS limits before importing numerical libraries, adds CORS, timing and memory middleware, retains health/predict, and registers RAG. `predict` reshapes supplied floats, computes mean/shape, and returns a fixed example detection; no ONNX inference session is used.

`DocumentLoader.ingest` checks size, sanitizes display filename, parses supported text, hashes content, checks duplicate under store lock, chunks/embeds outside the store transaction, checks duplicate again under lock, durably saves `doc_<uuid>.bin`, and publishes vectors. The double check handles concurrent uploads while avoiding expensive model work inside the first dedup lock.

`SentenceTransformerEmbeddings` lazy-loads CPU model under an RLock after memory preflight, with `trust_remote_code=False`; embeds normalized float32 vectors. Chunking uses LangChain splitters and preserves document/page/row metadata. PDFs without extractable text need external OCR; DOCX expanded size is bounded; CSV headers/row lengths are checked.

`FaissStore` implements LangChain VectorStore with `IndexFlatIP`. Normalize vectors so inner product equals cosine similarity. A file lock serializes reload/read/write transactions across workers. Each immutable generation contains native FAISS vectors and JSON metadata with checksum, model identity, dimension, chunk configuration and counts; CURRENT publishes a generation atomically after fsync. It uses no pickle. Deletion rebuilds remaining vectors and publishes a replacement; routine saves delete older regular generations. Rebuild retains explicit backups.

`Retriever` returns empty before embedding for an empty store, otherwise embeds query, reloads, filters by document IDs/threshold and returns `Hit(score,rank)`. Filtered queries search all chunks to avoid dropping eligible results. Optional CPU `Reranker` orders pairs with a cross-encoder but preserves original cosine scores; logits are not calibrated confidence.

`RAGPipeline` builds a LangGraph state: retrieve → optional rerank → bounded JSON context → generate or insufficient-evidence branch. Blocking parse/embed/store operations run off the async event loop where the API path requires it. Generation uses `ChatOpenAI`; configured base/model are required before the graph runs, even if it would have no evidence. `validate_citations` accepts only included source IDs and rejects absent/unknown citations. Valid IDs do not prove each claim logically follows from a passage.

`backend/app/rag/api/dependencies.py::get_pipeline` checks enabled status and creates one lazy pipeline per app worker under an RLock. RAG health reports not-initialized/empty/ready/unavailable storage without forcing model loading. `backend/app/rag/evaluation/metrics.py::recall_at_k` computes unique relevant IDs among first k divided by all relevant IDs (zero for an empty relevance set); `reciprocal_rank` returns 1/rank of first relevant hit or zero. These evaluate retrieval, not generated-answer truth. `backend/app/rag/rebuild.py::rebuild` locks the store, verifies original upload hashes, re-parses/re-embeds in staging, preserves IDs/explicit backup, and swaps CURRENT only after success; stop workers to avoid mixed model settings.

5.10 Process names and SQL hook integration

`proctitle.set_name` writes `/proc/self/comm` (15 characters), falling back to thread-level prctl. `set_title` can rewrite argv/environment memory, so only undiscovered runner/agent/watchdog processes use it. Plant/helper discovery depends on original command line and environment; their short names change without rewriting those inputs. A worker-thread prctl fallback can name only that thread, explaining sandbox-related naming test failures.

`sitecustomize` is loaded by Python from the injected PYTHONPATH. It names supported `fsmes` components and loads a dormant `Hook` only in configured database-client commands. The up-

stream source checkout remains unchanged, but runtime behavior is instrumented. “Unmodified target source” does not mean no lab code executes in the process.

6. Classes, APIs, schemas, configuration, and dependencies

6.1 Experiment contract and meaningful fields

```
name: api-process-outage-10s
category: infrastructure
target: {plant: bottling, component: api}
fault:
  type: process_outage
  parameters: {duration_s: 10, signal: KILL}
baseline_s: 20
workload: baseline
recovery: {timeout_s: 60}
scoring: none
```

`Experiment` stores name/description/category/steps/application/workload/baseline/preconditions/expected measurements/cleanup/recovery/scoring/source. `Step` stores target, fault type and parameters. A definition must use either top-level target+fault or nonempty steps, not both. Names match `[a-z0-9][a-z0-9._-]{0,63}`. Categories are `application` or `infrastructure`. Unknown top-level/fault keys are rejected; Boolean values cannot impersonate integer baseline values.

`preconditions` and `expected_measurements` are recorded metadata. They do not cause arbitrary named checks or enforce measurement existence. `cleanup` is description text, not a shell command. `scoring` is not a generic plugin dispatcher: the controller’s category selects actual harness scoring. Use built-in implementations to understand real behavior, not YAML prose alone.

Application settings include duration in **line seconds**, speed, seed and measurements. Speed compresses replay time; duration 1500 at 20× is about 75 seconds replay plus startup/scoring overhead. Fault windows must fit the scenario, event limits and policy.

6.2 States versus results

States: pending, preflight, baseline, injecting, fault_active, cleanup, recovering, verifying, completed, rejected, aborted, failed. `TRANSITIONS` permits sequential reinjection from appropriate cleanup/verification states. Terminal states have no outgoing edges.

Verdicts include `recovered`, `recovered_with_data_findings`, `not_recovered`, `rejected`, `rejected_preconditions`, `dry_run_ok`, `injection_failed`, `unsafe_cleanup_failed`, `aborted`, `error`. A completed run may have not recovered; dry run deliberately uses rejected terminal state with successful dry-run verdict. CLI codes: 0 success/recovered/dry-run, 3 recovered with findings, 2 refusal, 1 failure; usage code 64 is also defined. `cmd_status` succeeds when API is up; `cmd_health` applies stricter all-machine/component health.

6.3 Data contracts to recognize

Structure	Key fields and use
`Decision`	allowed, reasons, normalized steps, estimated wall seconds
`HttpResult`	ok/status/latency/body/error; transport failure carries unavailable body
`Check`	name/ok/detail; cleanup and removal are distinct checks
Lease	experiment ID/name/controller/watchdog PID, start/deadline/state/records
Injection record	fault type and target-derived recovery information, helper/old PID/config/limits as relevant
Sample	time/phase/API/machines/auth/floor/tag IDs/orders/component stats/database/cgroups/host
Recovery report	recovered, duration, timeout, stable sample count, first-stage timings, stages, timeline

Structure	Key fields and use
Agent data	Role, ToolCall(id/name/arguments JSON), Reply, AgentRun with rounds/transcript/output
RAG schemas	Document, Chunk, Hit, QueryRequest, Citation, QueryResponse, ingestion/retrieval/health responses
RAGState	request/hits/context/included/answer/citations shared between graph nodes

Read `Controller._new_result` for full result shape and `MetricsAdapter.sample` for normalized observations. Optional values must remain null, not synthetic zero.

6.4 Configuration and active dependencies

`Settings.path` resolves configured relative paths against root; pack TOML supplies actual host/port. `CHAOSLAB_ROOT` and CLI `--root` choose lab root; `CHAOSLAB_VENV` can override fsmes executable search. Fleet registry controls plant packs and data paths. The adapter sets absolute upstream config locations because relative files otherwise resolve from runtime work directory.

Reader account is `ANALYST`; password can come from `FSMES_ANALYST_PASSWORD`, with a local-lab default in configuration. The adapter caches a login token, drops it on authenticated 401, and retries login on a later request. It does not dump credentials. Hosted agent credentials are Mistral/Groq-specific; backend `LLM_*` settings are separate.

Representative bounds in `safety-policy.yaml`: general hold 60 seconds, baseline 5–60, recovery ≤90, estimate ≤600, maximum three steps but compound disabled, simultaneous target policy one; CPU pressure ≤80% and 30 seconds; DB delay ≤1000 ms and 30 seconds; resource duration ≤30, memory ≥64 MiB and CPU ≥5%; workload ≤5 requests/sec; application max 3600 line seconds, speed 1–30, one scripted event. Fault metadata and helpers impose additional bounds. `max_simultaneous_faults` documents policy intent; current controller achieves sequential execution rather than a general concurrency scheduler.

Backend `RAGSettings` loads `.env` from backend working directory, ignores extra fields and hides input in settings errors. Defaults: RAG enabled, `data/rag`, MiniLM-L6-v2, chunk 1000/overlap150, paragraph strategy, batch32, top-k5, context12000 chars, max25 MiB, optional threshold/reranker. Overlap must be below chunk size; reranking requires model. Model identity/chunk changes require rebuild. `LLM_BASE_URL`, `LLM_MODEL`, `LLM_API_KEY` do not configure chaos agents.

6.5 HTTP surfaces and ports

Surface	Routes / default ports	Boundary
Native MES	`/health`, `/metrics`, `/auth/login`, authenticated equipment/workorder/KPI routes; 8010	Adapter reads using read-only analyst
OPC replay	OPC TCP 4841	Telemetry server; not HTTP
Lab exporter	`/metrics`, `/healthz`; 9108	Loopback read-only monitoring
Optional Prometheus/Grafana	9090 / 3000	Separate local services
Application harness	API8100 / OPC4900	Ephemeral target, fixed-port constraints
Backend	`/health`, `/api/v1/predict`, `/api/rag/documents/upload`, list/get/delete documents, `/api/rag/retrieve`, `/query`, `/health`; 8000	Independent FastAPI; no lab-control routes
Upstream Compose	host8000→nginx80→api8000; mockERP8001	Alternative topology; internal Docker DNS

Loopback ports here describe internal validation/development, not published cloud preview links. Sharing a machine requires checking collisions, especially backend/Compose port8000 and harness fixed ports.

7. End-to-end execution walkthroughs

Detailed symbol chains are indexed in EXECUTION_FLOWS.md. These walkthroughs explain why each step matters and what to inspect when it fails.

7.1 Fresh setup and startup

From the root, bashscripts/setup.sh checks Python/venv/Git, clones upstream if absent, switches to the pin if available, creates .venv, installs requirements/editable target, and invokes prepare/check. The script can warn rather than fail when the pin is unavailable, so reproducible setup must separately verify actual upstream HEAD equals UPSTREAM_COMMIT. Do not blindly rerun against a user-modified dependency checkout that would be switched by setup.

Lifecycle.prepare creates directories, copies the bottling pack without generated output, generates replay with fsmessim-generate, writes fleet registry, initializes DB only if absent, and preserves existing results. The installed target's config files are referenced absolutely. Re-running setup here reported "nothing to do" for preparation and passed dependency checks with normal process visibility.

Lifecycle.start checks resources, initialized DB and absence of detected components, calls the upstream supervisor, then waits for healthy API/machines/processes. --with-exporter starts monitoring exposition afterward. First floor activity can lag startup; lifecycle health is less strict than full recovery because it does not check recent floor activity and advancing ingestion. The verified restart was healthy after about five seconds; current live PIDs are transient and must not be copied into future startup assumptions.

7.2 No-fault infrastructure baseline

baseline-observe.yaml requests 20-second baseline and 20-second no-fault hold. NoFault participates in the lifecycle without disturbing a resource. The sampler observes health/authenticated summary/ingestion/processes and the workload makes bounded reads. Cleanup reports nothing injected. Recovery still checks manufacturing evidence, followed by a post window.

Verified run 20261010T190848Z-baseline-observe-e3835a returned recovered, cleanup true, six connected machines, authenticated reads, advancing tag IDs and SQLite quick check ok. It proves the observation/recovery path functions with a healthy target; it does not prove a destructive injector. Its 0.0-second recovery value is possible because a stable healthy sequence starts immediately after cleanup and duration records that sequence's first sample.

7.3 API crash through direct CLI

cmd_run loads api-process-outage-10s.yaml, controller validates/gates/preflights, takes baseline, persists intent, calls ProcessOutage.inject to send SIGKILL to detected API, holds ten seconds, then restarts through spawn_component. Only the API is targeted. The OPC agent may continue storing data while API-derived metrics become unavailable; the floor may experience request failures.

Cleanup detection checks the new process exists. Recovery then requires API response, authenticated summary, all machines connected, recent floor check and advancing ingestion. Quick check and tag monotonicity assess data consistency. Inspect fault_injection.steps to prove injection, old/new process observations for restart, samples/logs for impact, and recovery stages for restoration. This live destructive demo was not run during the review; the mechanism is implemented and covered by component tests.

7.4 Same experiment through the Injector

python-mchaoslabagentinjectapi-process-outage-10s invokes commands.inject; it builds MCP server/toolbox plus chat client. Prompt directs discovery → plant/resources → validation → dry-run → real-run approval → wait. Each tool call is runtime-filtered to the role. A real submission calls ask_terminal, which uses the controlling terminal, not piped input. --approve is advance approval, not a policy bypass.

Once accepted, MCP `run_experiment` returns an ID while a detached runner executes the same controller. The model waits using `wait_for_experiment`, then reports JSON. Transcript stores exactly which calls occurred; final text is model-generated. If tools say unknown temporarily, diagnose the asynchronous startup window rather than instantly assuming experiment disappearance. If waiting repeatedly fails, emergency stop is the prompt's recovery instruction; it is not an automatic timeout action in the generic MCP waiter.

7.5 Analyst explanation after a run

`agentanalyze<id>` resolves a completed result and provides read-only evidence tools. Prompt asks recovery, metrics, logs, scoring when application-layer, a compatible recovered baseline, and current plant health. The model may disagree with the controller only with cited evidence, but the runtime does not validate logical entailment. It does verify a claimed baseline appears in actual comparison calls and derives emergency-stop usage from actual calls in injection output.

Read `agent-analysis-*.json` alongside `result.json`, not instead of it. An unsupported baseline claim becomes `baseline_compared_unverified` with baseline null. An empty/invalid final dictionary or a max-round stop is not a reliable assessment. Hosted calls were not verified here because the optional provider credentials were not supplied.

7.6 Application breakdown and scoring

`sim-machine-breakdown-rd.yaml` maps to a simulator down event, not SIGKILL. `_run_application` reads speed/seed/duration/measures, calls each simulator fault's `event`, writes TOML, and enables watched-state evidence by adding latency if no requested measure enables the watcher. `ScoringAdapter.run` calls `fsmeslabrun`, supervises stop/timeout and captures console output into the current run.

The harness creates a separate simulated plant/recording/DB. `derive` compares booked units against ground-truth replay bands, parses downtime detection/misclassification, OEE and state looks. After teardown, `_application_recovery` requires post-event evidence, successful answered operations and consistency. If the last look remains down/stopped/changeover, or no look exists after the fault window, recovery is not established. Wall-clock recovery duration remains null; speeding a simulation is not speeding a real process outage.

7.7 Backend upload, retrieval and answer

Upload a supported document to `/api/rag/documents/upload`. Route validates extension/size while reading chunks, then offloads `loader.ingest`. Parser returns LangChain documents; chunker preserves source metadata and UUID chunk IDs; model generates vectors; store normalizes/publishes snapshot. A repeated content hash returns deduplicated metadata, not additional indexed copies.

`/api/rag/retrieve` uses normalized query vectors, top-k/document filter/cutoff and returns chunks with cosine scores/ranks. `/api/rag/query` checks generation configuration, runs the graph and sends bounded context to chat model. Empty eligible context returns insufficiency only after configuration prerequisite passes. Nonempty answers require valid included citations. Unknown/absent citation produces 502 rather than inventing provenance. An embedding/model download failure is 503, not an empty-but-successful retrieval.

7.8 Evidence retrieval and comparison

`chaoslabshow<id>`/MCP `get_result` read a stored record; `report` formats it. `get_metrics` can include sample rows with truncation metadata and DB/resource-specific evidence. `get_logs` returns excerpts and audit. `compare_runs` uses two stored records; it does not rerun them or discover causation automatically. CSV export flattens selected fields. Save complete JSON/samples when a detailed debugging claim depends on evidence that summaries omit.

8. Fault mechanisms, approval, limits, cleanup, and recovery

8.1 Infrastructure mechanisms

Fault / implementation	Effect and injection path	Removal / cleanup proof
`none` / `NoFault`	Observe without disturbance	Nothing injected; recovery still observed
`process_outage` / `ProcessOutage`	KILL default or TERM to one allowlisted component; bounded hold	Restart absent component with original plant environment; verify detected live component
`process_freeze` / `ProcessFreeze`	SIGSTOP suspends one component without killing	SIGCONT; verify not stopped. A gone process can mean freeze removed but service still needs recovery
`cpu_pressure` / `CpuPressure`	Save affinity, choose permitted CPU, start duty-cycle burner, pin API to same CPU	Stop burner and restore affinity; independent check verifies both
`sqlite_write_lock` / `SqliteWriteLock`	Helper acquires `BEGINIMMEDIATE`, confirms LOCKED, holds without changing rows	Stop/expire holder, rollback, prove new write lock acquirable
`database_latency` / `DatabaseLatency`	Activation enables per-statement SQLAlchemy sleep in API/OPC agent	Remove activation and verify injected counts stop after settle/flush windows
`resource_limit` / `ResourceLimit`	Prepare component in delegated scope; reset helper before writing kernel limits	Restore original limits, stop helper, restart if killed, verify files and component

CPU pressure is contention, not a precise CPU quota. `memory.high` induces reclaim/throttling; `memory.max` can OOM-kill. CPU cap uses quota/period: 20% of one CPU corresponds to `cpu.max=20000 0100000`, not 20% of all host CPUs. Memory units are MiB converted to bytes. Host swap cap and component swap setting are distinct.

The SQL hook matches exact SQLite path and selected commands. Activation includes plant, database realpath, boot ID, token and monotonic expiry; invalid/expired/other-boot activations do not enable delay. Means separate observed/native/injected timing; fixed histogram percentiles are bucket upper bounds. Already-sleeping SQL may finish its one delay after the window expires. It is not packet loss or a proxy-induced database round trip.

Unsupported fault specs explicitly refuse `database_outage`, `api_latency`, `memory_pressure`. Removing a SQLite file risks stray new DB/data loss; API delay would require a topology-wide proxy rather than delaying only lab calls; unbounded memory consumption is disallowed. Historical reason strings about installed Docker/RAM may be stale, while unsupported policy status is current. Do not claim a discovered-but-unsupported fault is executable.

8.2 Simulator mechanisms

`SimDown`, `SimChangeover`, `SimMicroStops`, `SimCounterReset`, `SimScrapBurst`, `SimStarve`, `SimBlock`, `SimDisconnect` emit upstream scenario events. Windowed events use start/end line time; instantaneous reset uses at-time; micro-stops have seeded interval/duration parameters. Scripted disconnect affects the observer endpoint while the line conceptually continues. `NoFault` is a control in either layer.

Fault support inventory differs from shipped YAML inventory: e.g. scrap/starve/block registry entries do not all have stored definitions. Use discovery output for exact supported parameter schemas; do not infer parameters from a similarly named event.

8.3 Approval and enforcement boundaries

Agent `needs_approval` triggers only for non-dry `run_experiment`. `ask_terminal` refuses absent terminal and defaults no; preapproval uses explicit CLI flag. Tool membership and approval are implemented in code, not merely model instructions. Schema/policy/lease/preflight apply after approval; approval cannot make a forbidden target valid.

Direct CLI/Python API and external generic MCP calls do not pass through the Agent callback. Their invocation is the caller's action boundary. A future remote UI/API must supply its own authorization/approval mechanism. Destructive MCP metadata alone does not guarantee a client will prompt.

8.4 Cleanup is a journal-based protocol with limits

Intent is written **before** side effects. The full record afterward contains helper PIDs/affinity/original limits/etc. `cleanup_records` reverses records and combines cleanup AND removed checks; exceptions yield failed checks. Controller, watchdog and emergency stop share it. Helpers clamp their own lifetime to 120 seconds; SQL hook independently expires. Layered defenses reduce dependence on one successful happy-path return.

Important caveat: an intent lacks some undo information produced during injection. A crash after changing affinity but before full record publication can leave insufficient recovery context even after burner expires. Controller clears records after some failed cleanup paths and releases in finally. Correct presentation: the project attempts bounded cleanup and verifies normal cleanup, but does not prove instruction-level transactional restoration under every crash. Fix design proposals are in §13.

8.5 Sequential versus simultaneous injection

`Experiment.steps` supports multiple **sequential** infrastructure faults. Controller cleans and verifies recovery between them; next injection does not intentionally overlap prior fault. `compound.enabled` is false by default; stored compound definition is rejected. Maximum three steps and application max one event remain separate constraints.

There is no implementation of an overlap scheduler, per-target locking graph or parallel fault orchestration. Atomic lease replacement is not reliable global mutual exclusion under concurrent acquire. Enabling a YAML flag alone is insufficient evidence of complete multiple-injection support. §13 explains an incremental safe path and explicitly separates a future simultaneous design.

9. Observability and how recovery is measured

9.1 Sampling and metrics

`MetricsAdapter.sample` timestamps before its sequential probes. It queries health; if API up, authenticated work-order summary, equipment states and raw Prometheus metrics. It adds process, hook, cgroup and host observations. `Sampler` runs periodically in a thread, carries phase labels and exposes snapshots for recovery; slow HTTP calls can reduce actual cadence. Sample time is not the timestamp of every individual probe.

`summarise` computes samples/duration, API success ratio, authenticated ratio among probed samples, latency distribution, minimum connected machines, max floor age, ID-based ingestion slope and longest stall, end quantities, component-alive ratios/RSS maxima and host extremes. Authentication ratio excludes unavailable probes during outage, so read it together with overall API availability. Successful-response latency excludes failed requests; low latency during an outage is not recovery evidence.

Ingestion slope = `(last_tag_id-first_tag_id)/elapsed_observation_time`. It is a proxy based on monotonic IDs, not a count of physical bottles or guaranteed uniform row spacing. During API outage, missing metrics are unavailable rather than a measured zero. DB/cgroup evidence is collected directly from local instrumentation and is kept separate from downstream API effects.

9.2 Five-stage infrastructure recovery

`RecoveryVerifier` defaults: three stable samples, floor age ≤45 seconds, ingestion reference at least three seconds old.

1. **fault_removed:** independent fault-specific removal check passed.
2. **service_available:** API status ok AND all configured components alive.
3. **operation_succeeds:** authenticated work-order summary succeeds.

4. **manufacturing_behaviour:** machine counts available and all connected; floor check recent; current tag ID greater than an older observation.
5. **data_consistency:** read-only SQLite `PRAGMAquick_check` ok AND latest tag ID not below baseline maximum.

Core service/operation/manufacturing checks plus removal must be true for three consecutive fresh post-cleanup samples. A failure resets the count. Duration records the **first** sample of the successful stable sequence relative to recovery-wait origin, not its third sample or time from original injection. Stage first-ok timings, stable sequence and final data check are distinct measurements.

DB latency/resource caps add `database_latency_restored` / `resource_limits_lifted` stages from direct evidence. Overall `recovered` requires all relevant criteria. Core success with failed data consistency produces `recovered_with_data_findings`; failed core behavior gives `not_recovered`; failed cleanup overrides recovery with `unsafe_cleanup_failed`. Sample history, not PID alone, explains the conclusion.

9.3 Application recovery evidence

`_application_recovery` reuses stage names with different evidence: scripted window ends and ephemeral teardown; completed score-producing harness with watched data; OEE/booking answered across stations; post-event states; booking within the harness band and no misclassified planned stops. It does not pretend there is persistent DB quick_check or an infrastructure recovery time. `duration_s` is null by design.

OEE means availability × performance × quality, but the MES intentionally withholds unsupported availability/OEE when observation coverage is insufficient. Null is an honest unknown. Station states sampled once per wall-clock look at accelerated speed can miss short events; absence of a detected fault alone does not prove it never happened.

9.4 Exported series, event flow and dashboards

Representative `chaoslab_*`: plant/component up, health latency, connected/total machines, floor check age, good/scrap quantities, state code, component RSS/CPU, host resource metrics, active experiment/state/start, latest verdict/recovery/cleanup and recorded-run count. Latency is seconds in exporter and milliseconds in sampler/report; avoid unit mix-ups.

MES series include `mes_tag_values_max_id`, audit max ID, work orders by status, equipment coverage/availability/not-observed seconds, plant information and optional replay factor. No exported MES request histogram or OEE performance series exists in this integration. The lab measures API latency itself and reads KPI API for OEE.

Prometheus scrape: exporter five seconds, MES ten; rules evaluate fifteen. Rate windows and delayed alert duration mean a ten-second fault can be clearer in per-second run samples than in Grafana. Alerts in `lab.rules.yml` exist, but no Alertmanager delivery setup or routed notification was verified. Historical “alerting not implemented” should be qualified as missing operational delivery, not absence of alert expressions.

Event flow is audit + sample timestamps + target JSON logs correlated by experiment ID/window. The lab does not consume MES `/line/events`, ERP outbox or MQTT as a live event bus. Log collector retains first bounded important lines and recent bounded info lines; excerpts are not the entire log. Redaction recognizes common secrets, not every sensitive string. No Loki deployment is implemented.

9.5 Measurement caveats to explain

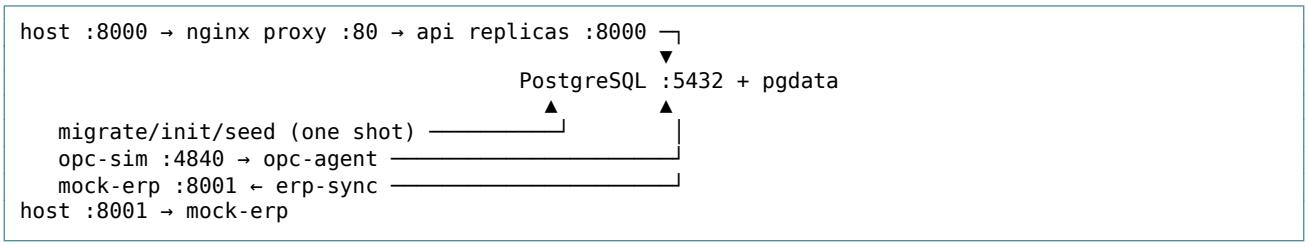
Component CPU `cpu_percent(interval=None)` on freshly discovered process objects can be an initial zero; do not use it as calibrated proof of no CPU load. Host sampler is another measurement. Integrity checks detect structural corruption, not lost work or correct quantity accounting. Process names are diagnostic labels, not safe process identity. `/healthz` on exporter is not MES recovery. A Grafana green panel cannot override failed removal/data evidence. Separate causal hypothesis, observation and inference in every report.

10. Docker, deployment, networking, and host/container boundaries

10.1 What Docker does and what exists here

A container packages an application's filesystem/process environment, while networking, PID namespaces and cgroups determine what it can see/control. Docker Compose declares services and dependency ordering. This lab's standard workflow uses native processes and SQLite and never requires a Docker socket. An available Docker binary does not mean the project runs in Compose.

The implemented Compose files belong to the **upstream MES dependency**:



`docker/Dockerfile` uses Python3.13-slim, copies source/config/licenses, installs `.[postgres]`, and defaults to API listening all interfaces. Same image serves different roles through Compose commands. `postgres:16-alpine` has health check and named data volume. `migrate` runs once before API/agent/ERP sync consumers; APIs do not race migrations. `proxy` mounts nginx config read-only and publishes host8000. API has no directly published port so replicas can scale. OPC sim binds all interfaces inside its network; agent connects using service DNS `opc-sim`.

Nginx uses Docker resolver `127.0.0.11`, five-second DNS validity and variable proxy destination a `pi:8000`, enabling re-resolution as replicas change. Shared secret key allows API replicas to accept the same signed sessions. The source comments state stateless token sessions; do not conflate shared database state with no state anywhere.

10.2 Boundary differences and safe implications

Concern	Native lab	Upstream Compose
Database	SQLite path	PostgreSQL service/address and persistent volume
Process identity	Same-user psutil environment/argv	Container namespace/identity; native discovery insufficient
Fault control	Signals and lab helpers	Would need explicit bounded container-aware adapter
Resource scope	Delegated user systemd component scope	Container cgroups/daemon ownership, different permissions
Networking	Loopback API/OPC plus fixed harness ports	Service DNS/internal ports, host published proxy/mockERP
Restart	Adapter fsmes process/env/PID file	Container lifecycle/health/replica semantics
Recovery evidence	Local process + SQLite + APIs	Container/service/replica/ PostgreSQL-specific checks needed

`localhost` inside one container means that container, not PostgreSQL or host. Killing one API replica behind nginx may show no user-facing outage if others serve traffic. Container-wide kill and one worker-process crash test different hypotheses. A SQLite lock fault cannot be reused as PostgreSQL outage. Do not mount the Docker socket into an LLM agent as a substitute for an allowlisted adapter.

Compose publishes ports without loopback host restriction and has trial credentials/secret defaults. It is not the same local-only security posture as lab. TLS termination, credential management, access control and real production readiness are not demonstrated by these files. Compose and PostgreSQL mode were not run here.

10.3 Independent backend deployment

backend/deploy.sh validates worker/resource inputs, installs CPU-only torch before requirements, writes a bosch-backend systemd service, enables/restarts it with Gunicorn/Uvicorn binding all interfaces port8000 and 120-second timeout. Default four workers is retained; use GUNICORN_WORKERS=1 for low-memory local embeddings. Each active worker owns a model; file lock shares on-disk index safely, not model memory.

scripts/run_constrained.sh launches a user-systemd scope with hard memory/CPU/task controls; resource_probe.py measures profiles. backend/app/os_limits.py::apply_os_limits sets numerical thread env vars before imports, permitted CPU affinity and niceness. memory_status reads tightest ancestor cgroup limit, working set excluding inactive cache, peak and OOM counters; middleware sheds heavy POSTs with 503/Retry-After. Model preflight rejects insufficient memory before model loading. These are safeguards, not guarantees against simultaneous allocation peaks.

Backend data/cache permissions and .env working directory must be correct. Set external writable RAG storage for read-only source deployment. Changing embed/chunk settings requires stopping workers, backing up/rebuilding, restarting all workers; locking does not fix stale model configuration in running workers. This service is neither lab controller nor an implemented Docker deployment of RAG.

11. Testing, debugging, failure scenarios, security, and trade-offs

11.1 Tests and verified scope

From root: .venv/bin/python-mpytesttests-q. Verified **222 passed, 4 skipped** with normal same-user process access. Four delegated-scope tests skipped because user systemd/cgroup app slice is absent. First sandboxed run: 219 passed, three name-related failures, four skipped. Removing the process-access restriction through supported execution mode made all non-scope tests pass; no application patch was needed.

Test file/group	What it examines
test_model.py, test_config.py, test_policy.py	Schema, required config, normalization, forbidden targets, limits and category separation
test_controller.py, test_api_contract.py	State/control outcomes, error/cleanup reporting, API submissions/ results/stops with stubs
test_faults.py	Real helper signals/locks/pressure and injected-fault unit behavior
test_database_latency.py	Exact DB matching, delay/expiry/counters, client loading, removal evidence
test_resource_limit.py	Kernel units and policy/cleanup; actual scopes conditionally skipped
test_recovery.py, test_metrics.py	Stable evidence, missing data and metric aggregation
test_watchdog.py	Dead/expired lease cleanup and experiment-ID binding
test_store.py, test_scoring.py	Persistence/resolve/CSV and harness score/state parsing with fixtures
test_workload.py, test_observability.py	Read traffic bounds, exporter and dashboard contracts
test_proctitle.py	Naming preservation and worker-thread process labels
test_agents.py, test_agent_injections.py, test_mcp_server.py	Fake model/transport, role restrictions, approval, tools, fault reporting
backend/tests/test_rag.py, test_os_limits.py	Deterministic embeddings/mocked LLM/parser/store/query and memory limits; not run here

Stubs/fixtures isolate deterministic invariants and failure modes; they do not prove every real integration. The no-fault live run verified ingestion/auth/data checks. Destructive demos, real application harness, real models, Compose and monitoring servers remain unrun here. For backend use its separate virtualenv, CPU torch and requirements-test.txt, python-mpytest-q, and pip check; avoid source changes to make tests pass.

11.2 Debug by lifecycle stage

Symptom	Likely boundary	Concrete investigation
Setup cannot clone	Git/network/pin	Read-only `gitls-remote`, proxy configuration/access; compare dependency HEAD with pin; never dump auth
Port reported busy after startup	Genuine collision or hidden processes	`/health`, process argv and ability to inspect environments; a restricted sandbox may hide ownership
API ok but components missing	psutil permissions/namespaces	`find_components` requires MES_PLANT_NAME environment; check AccessDenied without printing values
Plant starts but no booking	Relative config paths/OPC	Check absolute overrides, replay out files, opc-agent JSON logs and tag IDs
`dry_run_ok` then refusal	Unchecked live prerequisite	Real preflight adds health/hooks/ scopes/lock checks; dry run is intentionally limited
Accepted ID initially unknown	Detached startup window	Read submission log/lease/current result; wait rather than submit duplicates
Fault has no observed effect	Injection versus measurement	Confirm injected step/record and direct helper/hook/cgroup evidence; API summary alone may be unavailable
Cleanup false	Restoration problem	Inspect cleanup+removed details; emergency stop; preserve records/logs; no recovery claim
Health returned, not recovered	Business behavior missing	Auth, connections, floor age, advancing tag IDs, data check and stable samples
Lock fault failure	Existing write transaction or helper startup	LOCKED handshake and write-lock-free precondition; avoid deleting DB
Delay fault refused	Hook not loaded/stale activation	Restart plant through lab; matching current PID stats; emergency stop activation
Resource caps refused	No delegated scope	`cgroups.available`; do not run host-wide caps to approximate missing capability
Agent call fails	Provider config/quota/transport	Presence of required binding, model/ base compatibility, response/retry; preserve tokens privately
Agent denied / loops	Approval or model instruction	TTY unavailable defaults no; inspect transcript; prompt discipline is not a hard retry latch
RAG upload 422	Parsing/content	Extension/signature/UTF-8/empty/CSV/ OCR; parser does not simulate OCR
RAG 503	Model/config/storage/memory	Lazy model cache, writable paths, CURRENT/model mismatch, lock timeout, memory guard
RAG 502 / 504	Citations/provider	Unknown/uncited response vs rejected provider vs timeout; retain expected HTTP distinction
Empty Grafana graph	Absent series/service/query	Exporter/MES scrape target, Prometheus readiness, missing measurements, retention; absence not zero

Use `scripts/health-check.sh` for readiness, not status command exit alone. Preserve underlying CLI exit status when piping/capturing. `inject-common.sh` direct wrapper can print a result ID despite a failing run code; inspect the JSON verdict. Never test a proposed fix by disabling assertions or converting missing evidence to success.

11.3 Confirmed source risks versus demonstrated failures

Source audit found no mutual-exclusion lock in `Lease.acquire`, incomplete undo data in early intents, clearing of failed-cleanup records, and no controller cancellation on watchdog timeout. These are inspectable implementation gaps, not claims that a live unsafe event was reproduced here.

Compound loop may stop after interim failure and later produce a final recovered verdict without requiring all planned steps ran. A safe report must compare planned versus executed steps. Sampler phase summaries combine all fault windows, so distinct effects can be obscured. Some descriptive preconditions are metadata only. Agent final JSON lacks strict schema validation; runtime neither mandates validate/dry-run order nor one accepted experiment nor permanent refusal after denied request. These are priorities for new code tasks, not changes made during this documentation task.

11.4 Security model and limitations

Implemented controls: target allowlist; no arbitrary PID/path/URL/shell in definitions; local plant/exporter; read-only MES analyst; bounded helpers; role-tool restrictions; approval for model-requested real runs; safe YAML; log redaction; embedding remote-code disabled; generated upload names and checksums; JSON metadata instead of pickle; exact CURRENT ID validation.

Limitations: local files/records assume trusted writer; checksums detect corruption, not a malicious replacement of both file and checksum. PID reuse and process environment access require care. Read-only model role does not prevent sensitive log/document text reaching a hosted provider. Prompts reduce injection risk but are not complete security boundaries. Agent/provider error text and transcripts may contain content needing protection. RAG broad CORS/no auth requires a gateway for controlled exposure. Compose trial secrets and all-interface bindings need separate deployment hardening.

11.5 Design choices to justify accurately

- **Native lightweight target:** lower installation/resource cost and sufficient components for process/SQLite experiments; sacrifices full PostgreSQL/container/network realism.
- **Pinned unmodified upstream source plus copied pack:** avoids fork drift and isolates data/config; runtime SQL hook is still instrumentation and must be acknowledged.
- **Deterministic policy below AI:** inspectable rejection independent of model reasoning; does not solve all OS/concurrency bugs.
- **Adapters:** centralize target knowledge; changing deployment requires explicit new semantics rather than pretending signals work everywhere.
- **JSON results and audit:** easy inspect/export without external services; weaker multi-writer/durable transactional guarantees than a database-backed coordinator.
- **Three stable samples:** less false recovery from one transient response; detection depends on cadence and can miss behavior between probes.
- **Unavailable values:** honest uncertainty; presentation must handle sparse series and incomplete comparisons.
- **MCP stdio/in-process:** reusable structured tools without network control server; remote UI needs a separately secured transport/application.
- **Separate AI analysis file:** preserves authoritative run evidence; model reasoning still needs validation and evidence review.
- **Local exact FAISS:** understandable retrieval and simple persistence; linear scans/full snapshots/filter scans and per-worker models limit scale.
- **Layered expiry/watchdog/cleanup:** resilient normal restoration paths; does not provide transactionally complete cleanup in every crash window.

12. Limitations and unfinished deliverables

12.1 Requirement and evidence gaps

No original problem statement or Day 2 requirements were found, so no claim of full Bosch deliverable completion is supportable. Referenced experiment/resource assessment docs are missing. Existing historical verification reports must not be mistaken for current checks. Main suite passed; scope tests skipped; optional stacks and destructive integrations remain unverified here.

12.2 Capability gaps

No implemented recommendation engine connects results to evidence-backed remedial suggestions. Analyst's `suggested_next_experiment` is only model output. RAG has no domain corpus or lab-agent integration. No HTTP lab control API, custom interactive frontend, Docker fault adapter, PostgreSQL fault integration, packet loss/partition/host network shaping, multi-plant orchestration, or simultaneous injection engine is implemented. OPC wrapper depends on missing external tool. Full ML prediction model is absent despite placeholder detection output.

Sequential infrastructure steps exist but disabled and need completion/reporting/race tests. App max-event remains one. User-systemd caps cannot run on this cloud host. Monitoring files include alerts, but notification routing/Loki/monitoring deployment are not verified. Licensing for the lab itself is not chosen in README; upstream and dependencies have their own terms.

12.3 Safety and measurement gaps

Lease mutual exclusion, durable intent/undo publication, cleanup-failure retention, watchdog cancellation, PID identity and compound completeness deserve implementation work before stronger safety guarantees. Host-memory gate is not the backend's cgroup-aware implementation. CPU stats can understate usage. API outage hides API-derived ingestion. SQLite structure and IDs are narrower than manufacturing truth. Fixed harness ports and compressed-time sampling limit concurrency and recovery interpretation. Hosted models can misstate results, truncate evidence or stop without usable JSON.

13. Future implementation plan

Everything in this section is a **proposal grounded in current code**, unless explicitly marked already implemented. No unavailable team requirement is silently converted into an agreed design.

13.1 Dependency-ordered foundation

Before expanding fault autonomy, fix coordination/evidence foundations: (1) real inter-process lease acquisition lock; (2) owner ID/generation checks on updates/releases; (3) persist enough undo data before each effect; (4) retain unresolved cleanup records; (5) cancel/fence overdue controller; (6) enforce planned/executed step accounting; (7) typed agent/recommendation output and evidence validation. Add adversarial concurrency and crash-window tests. These changes support all three requested future topics.

13.2 Recommendation engine

Problem and existing pieces

A run says what happened, but a user also needs an explanation of weak behavior, evidence-supported improvement candidates and the next safe experiment. A recommendation engine should separate **diagnosis**, **remediation proposal** and **next-test proposal**. It should not mistake a healthy process after cleanup for no business loss or automatically apply an LLM suggestion.

Already present: `ResultStore` records/comparison; `LabApi` evidence accessors; Analyst findings and suggested-next-experiment field; generic RAG parsing/retrieval/citation infrastructure. Missing: integration, domain corpus, deterministic evidence package, typed recommendation contract,

ranking/confidence handling, validation/evaluation, user workflow and any approved remediation executor.

Proposed architecture and contract

```
finished result + compatible baseline + samples/logs
          → EvidenceBuilder (local, deterministic)
          → optional retrieval of approved project/runbook passages
          → read-only recommender (rules + optional LLM)
          → schema/evidence/policy validation
          → separate recommendation.json + human review
          → explicitly approved next experiment via existing controller
```

Proposed types, not current schemas:

```
EvidenceRef = {"run_id": str, "file": str, "field_or_window": str}
Recommendation = {
  "id": str, "version": 1, "run_id": str,
  "kind": "diagnosis|remediation|next_experiment",
  "finding": str, "evidence": list[EvidenceRef],
  "action_proposal": str, "risk": str,
  "uncertainties": list[str], "expected_effect": str,
  "verification_plan": list[str],
  "experiment_definition": dict | None,
  "policy_validation": dict | None,
  "status": "proposed|rejected|reviewed"
}
```

Inputs must include layer/deployment/target/workload/seed/version/cleanup/recovery and missing measurements, not only overall verdict. RAG sources should be approved project docs/runbooks with revision identity. Output cites exact run fields/time windows and optional document chunks. Confidence should mean explicitly defined evidence sufficiency, not raw cosine similarity or an invented probability.

Integration options: a new read-only `LabApi.recommend`/MCP tool within analyst scope, or a separate service reading validated evidence. Neither is implemented. If using backend RAG, first define its network/auth/data contract; do not assume importing backend modules into lab can reuse incompatible dependencies safely.

Implementation order, safety and demonstration

1. Define types and evidence reference resolver; reject invalid/nonexistent references and cross-run/deployment mismatch.
2. Build compact evidence packages preserving nulls, failures, cleanup safety, baselines and per-step facts; whitelist/redact content sent to model.
3. Implement deterministic initial rules (e.g. cleanup failure → restoration investigation, available service with stalled ingestion → observer path investigation). Rules produce hypotheses, not guaranteed root causes.
4. Assemble/version domain corpus; expose retrieve-only integration; test source selection and sufficiency before generation.
5. Add bounded model generation with schema validation, safe tool permissions, error handling and grounded citations. Handle unavailable provider by retaining deterministic analysis.
6. If proposing a next experiment, parse with `model.from_mapping` and validate through existing `PolicyValidator`; attach rejection reasons without silently weakening it.
7. Save recommendations separately and expose review/read tools. Execution must be a separate explicit approval action, with controller duration/lease/cleanup/recovery requirements unchanged.
8. Evaluate recommendation quality on labelled fixtures and held-out runs; compare to rule-only baseline; track supported claims, actionable verification, rejection accuracy and uncertainty handling.

Safety: no policy edit, arbitrary shell, container socket, process control or production remediation tool in recommender. Unsafe cleanup blocks further experiments until resolved. Approved tests remain allowlisted/bounded; recommendation acceptance does not imply injection approval. Redaction and network disclosure requirements apply before hosted calls.

Tests/demonstration: fabricated citation rejected; missing metric remains unknown; wrong baseline rejected; malicious runbook cannot cause execution; unsupported fault proposal cannot run; provider timeout yields stored inconclusive output; valid recommendation links to actual measurements and proposes a policy-valid dry run. Demonstrate one observed failure, its evidence-backed hypothesis, a reviewed code/config improvement in a separate task, and compatible before/after validation. Avoid claiming causal improvement from one noisy run.

13.3 API-crash injection

Problem, current implementation and scope decision

Crash injection tests whether loss of the API disrupts reads/operator activity and how quickly useful behavior returns. **Native MES API crash already exists:** `ProcessOutage`, API allowlist, `api-process-outage-10s.yaml`, original-environment restart and staged recovery. Do not propose "implement SIGKILL" as if absent.

New work can improve this path's evidence/approval/UI, or add a different target: generic backend worker, complete Gunicorn service, container, or one replica. Those targets require explicit adapters and approved target policies. Killing one worker under Gunicorn automatic respawn is not equivalent to holding the entire API down ten seconds; choose hypothesis and supervision semantics first.

Proposed inputs, outputs and integration

Retain symbolic target, signal choice, bounded duration, baseline/workload and recovery timeout. Proposed evidence fields: observed old/new process identity including create time, signal timestamp, first failed request, failed-operation window, restore intent/process spawn time, first successful request, stable recovery start, floor/ingestion effects and DB consistency. Separate **command accepted**, **process actually exited**, **user-visible outage**, **restart occurred**, **recovered**.

The definition/controller/fault interfaces can remain unchanged for native API. Enrich `ProcessOutage` records and controller measurement fields in a separate coding task; add an API-focused wrapper only if it uses shared definitions/policy/approval. Proposed new deployment adapter should have typed target identity and restore recipe rather than user-provided shell command or arbitrary container name.

Dependency-ordered work and safeguards

1. Confirm native versus backend versus replica/service scope; document other dependencies and automatic restarters.
2. Fix lease/intent/fencing foundation. Persist restart specification and stable target identity before signaling.
3. Add probe workload and event timing that can distinguish no user-facing impact from a failed injection.
4. Use existing native `ProcessOutage`; for new target write adapter and fault-specific prerequisites/restore/removal checks. Avoid modifying MES application routes to throw artificial exceptions as a substitute for OS crash.
5. Gate model/UI-triggered real run with immutable definition approval; revalidate at execution. Keep duration bounded and watchdog/helper ownership independent of the API being killed.
6. Restore only the allowed target and original configuration, verify PID identity and process health, then staged application behavior/data evidence. On failure retain undo records and refuse recovery claim.
7. Save complete evidence and compare with a same-workload baseline; add presentation/report support.

Tests: model approval denied does not signal; policy rejects arbitrary PID/URL; one API crash leaves OPC/other processes untouched; signal failure and restart failure generate correct outcomes; controller dies immediately after signal and watchdog restores; lost/in-flight transaction is reported as data uncertainty; new PID observed with correct settings; health-only success does not satisfy recovery. Use isolated target and bounded integration, never external production API.

Success demonstration: a live baseline; approved native API outage; measured API failure window; original configuration restart; authenticated/floor/ingestion recovery across stable samples;

integrity evidence; logs and precise limitations. For multiple replicas, success may be preserved availability rather than visible outage—state the hypothesis rather than forcing a failure interpretation.

13.4 Multiple injections

Problem and what exists

Sequential disturbances test recovery before a second fault; simultaneous disturbances test interacting failure modes and increase blast radius. Current `steps`, compound definition, policy switch, cleanup and intermediate recovery implement the sequential skeleton. Default compound disabled, maximum three steps, one active fault intent. Concurrent overlap is not implemented; application max events is one.

Proposed sequential contract and architecture

Use current normalized steps and add per-step execution status, outcome, window timestamps, measurements, cleanup and recovery, plus aggregate planned/executed/completed count. Proposed status: pending, preflight, injected, cleaned, recovered, failed, skipped_due_to_prior_failure. Full result must never say the whole plan succeeded when a planned step did not run.

Preserve one owner lease for entire plan. Prepare each target only when needed; preflight after prior recovery because prior fault may alter readiness. Separate phase label or step index so measurements cannot merge API and OPC outages indistinguishably. Baseline remains clearly defined as whole-plan or refreshed per-step; that choice is a proposal to settle before implementing comparison.

Implementation order: fix acquisition ownership/fencing; add per-step identity/outcome schema; require all planned steps in aggregate success; recheck health/preconditions before every step; retain full cleanup journals; tag samples step+phase; bound total wall estimate including intermediate recovery/post windows; enforce one immutable approved plan; then deliberately enable compound policy in an isolated test configuration. A model must not substitute extra steps after approval.

Simultaneous design requires new work

If documented requirements later require overlap, propose an explicit schedule rather than reinterpret sequential steps: start offset/duration, target resource set, maximum active count, dependencies, conflict matrix and full rollback order. Validate overlapping windows, shared resources and full worst-case duration before execution. Examples of conflicts: DB write lock plus SQL delay may prevent distinguishing causes; resource cap plus outage on same PID complicates restart/restore; stopping API and OPC observer together destroys much read evidence.

A single orchestrator should own bounded tasks and one plan journal; each task reports intent/effect/removal. It must coordinate reverse dependency cleanup, handle partial start failure, cancel all effects on unsafe condition, and rely on independently expiring helpers. Watchdog must know every outstanding effect before it starts. This requires reviewed policy changes, more robust resource observation and target-specific recovery criteria; spawning parallel calls to current LabA `pi.submit` is unsafe and incorrect.

Safety, tests and demonstration

Sequential requirements: operator approval covers exact plan; policy allows it; bounded holds and total wall; each cleanup verified; second step prevented if first does not recover; emergency stop works in every step; incomplete plan never recovered aggregate; results retain skipped reason and unresolved restoration details. Simultaneous requirements additionally need cap on active effects, resource/conflict validation, independently tested combined recovery and approval displaying full combined blast radius.

Tests: concurrent lease attempts allow one owner; denial starts zero steps; invalid later step rejects plan before first injection; failed first cleanup blocks second; failed intermediate recovery reports incomplete; controller/watchdog death during each journal boundary; combined fault expired/partial-start cleanup; per-step metric segmentation; total budget enforcement; no changed

plan after approval. Demonstrate sequential API→OPC outage with evidence of full recovery between, then plan accounting and all cleanup checks. Keep overlap disabled until that foundation works and any original requirement actually demands it.

13.5 Broader delivery sequence

After foundational fixes, deliver native API evidence improvements, then safe sequential plans, then read-only recommendation engine with evaluation, followed by explicitly requested new adapters/deployments. Update audit and execution flow per release. Add operational monitoring delivery, missing reports/licensing and real backend evaluation when relevant. Record requirement provenance when original Bosch/Day 2 documents become available; no feature roadmap substitutes for their acceptance criteria.

14. Technical interview preparation and glossary

14.1 Beginner questions with grounded answers

1. What is the project in one minute? A bounded local chaos lab around a simulated FactorySemantics MES. It validates proposals, perturbs allowed software resources or scripts manufacturing scenarios, removes faults, checks recovery from application evidence and saves results. Two LLM agents can drive tools or analyze evidence. A separate RAG backend exists but is not integrated into fault recommendation.

2. What is an MES and why is API health insufficient? An MES turns machine observations into work orders, quality, downtime and KPIs. The API can answer while OPC data is stalled. Recovery therefore needs authenticated reads, connected machines, recent floor actions, advancing ingestion and consistency in addition to service availability.

3. What does a baseline do? It records behavior before disturbance using the same target/workload. `baseline-observe` uses no fault so comparison can distinguish normal variation from impact. A mismatched baseline (different layer/speed/workload) may be misleading.

4. Does the lab require Docker or an API key? Native Python/SQLite workflow requires neither. Hosted Injector/Analyst needs provider key and network; Prometheus/Grafana are optional. Upstream Compose and RAG are independent alternatives/workflows.

5. Where is the main entry point? `.venv/bin/python-mchaoslab` enters `__main__` then `cli.main`. Backend runs `app.main:app` from backend directory. These are different applications.

6. What does MCP do here? It exposes 21 named structured tools over stdio; model clients discover schemas and invoke them. Tools call LabApi/lifecycle/resources, which reuse deterministic policy/controller. MCP is not a substitute for approval or the experiment engine.

7. What is the difference between an outage and a freeze? `ProcessOutage` sends KILL/TERM and later restarts a process. `ProcessFreeze` sends STOP then CONT; process exists but cannot make progress. Removal/recovery observations differ.

8. Are the returned detections actual ML predictions? No. Backend `predict` computes shape/mean and returns a fixed example detection. ONNX Runtime being installed does not mean an ONNX model is loaded.

9. What does null mean in a result? Unavailable or not applicable. It is not zero and cannot be filled in. OEE can be withheld because observation coverage is insufficient; application recovery duration is null because no comparable infrastructure wall-time metric is defined.

10. Have all Day 2 requirements been met? That cannot be determined: original Day 2/problem documents were not found. Describe implemented and verified evidence, and explicitly identify missing acceptance criteria.

14.2 Intermediate questions

11. Walk through an infrastructure run. Parse/schema → policy → host gate → lease/watchdog → real health/fault preconditions and preparation → sampler/workload baseline → persisted intent

→ inject/hold → cleanup and removal → stable recovery → post window → summaries/logs/final result → lease release. See controller and flow §2 for failures and compound nuances.

12. Why is intent saved before injection? To give cleanup another actor enough information if controller disappears during a side effect. The current generic intent is incomplete for some effects; full undo must eventually be persisted before effects to close crash windows.

13. What happens when dry run succeeds but resource caps cannot run? Dry run only validated schema/policy and host resources; no live fault-specific checks occurred. Real preflight detects missing user-systemd scope and refuses before injection.

14. How does approval work? Agent `_dispatch` checks real `run_experiment` with approver. Terminal yes or explicit preapproval allows proceeding; no terminal defaults no. Policy still applies. Direct CLI/API/external MCP calls do not inherit this approver.

15. Can the Analyst inject a fault? Its runtime role list contains only read tools; an attempted run call is refused even if a model asks. Saving analysis is performed by command orchestration afterward, not a model write tool. The Analyst cannot edit policy.

16. Is one experiment per Injector task a hard guarantee? No. It is a prompt instruction. Runtime enforces tool membership and approval per real run, but has no accepted-run counter. A future stricter agent should add a plan/run budget and refusal latch.

17. How is SQL latency different from a DB lock? The lock helper blocks competing SQLite writes through a transaction without changing data. SQL hook sleeps before cursor statements in selected clients/exact DB. They test contention versus execution delay, with different evidence and cleanup.

18. What prevents SQL delay persisting forever? Monotonic deadline, boot identity and hard window/delay checks inside target hook, independent of controller/watchdog. Cleanup removes activation and verifies no new delayed statements after settling. A currently sleeping statement may complete one delay after expiry.

19. How does resource limiting work? A component is restarted into a delegated user scope before baseline. Reset helper starts before kernel limit files are written. Cleanup restores limits/restarts if OOM and verifies. It applies to component scope, not host; unavailable without delegation.

20. What does recovered actually mean? Verified removal; available API/components; authenticated operation; connected/recent/advancing manufacturing evidence over three consecutive samples; and integrity/ID consistency, plus fault-specific restoration stages. Data findings produce a separate verdict rather than an unconditional pass.

21. Why can recovery duration be zero? It records the start of the first stable successful sequence relative to cleanup recovery wait. No-fault baseline can immediately have healthy samples; it does not imply zero disturbance downtime or zero observation overhead.

22. Where do results live? Unique run directories under results with JSON, samples, logs and harness artifacts; audit/lease/transcripts under runtime/state. JSON publication is atomic replacement. Completed-run listing requires a result file. Generated results are ignored by Git.

23. Why are there two dashboards? MES screens show operator/manufacturing state; Grafana shows metrics collected by Prometheus. Backend API is separate and no custom lab frontend is implemented. Dashboards do not determine controller verdicts.

24. How does RAG answer a question? Query vector retrieves normalized cosine hits, optionally reranked; bounded evidence JSON feeds chat model; citations must match included chunks. Empty context yields insufficiency after generation configuration check. Retrieval alone needs no hosted LLM key.

25. How are duplicate uploads and concurrent writes handled? Hash checks under store lock before and after embedding, then generated durable upload name and snapshot publication. Every store transaction reloads under cross-process file lock. It avoids stale worker state but serializes operations and assumes local filesystem.

26. Why normalize vectors? Inner product of unit vectors equals cosine similarity. FAISS IndexFlatIP then performs exact cosine ranking. Similarity is not probability that an answer is correct; reranker logits are not substituted as confidence.

14.3 Advanced questions and failure cases

27. Is atomic file replacement a lock? No. It prevents readers seeing partially written JSON, but two processes can both read a free lease and both replace it. `lock_path` is unused. Real inter-process acquire/update ownership and fencing are needed for strong one-run guarantees.

28. What happens if the controller dies after injection? ID-bound watchdog reads persisted records and cleans them; helpers expire independently. This is robust in normal recorded paths but minimal-intent crash windows may miss affinity/original/helper details. Do not claim transactional restoration until those records are complete before effects.

29. What happens if watchdog deadline passes while controller is alive? Watchdog cleans/releases but does not kill/fence controller. The controller could continue. Future cancellation and ownership generation checks must prevent late side effects/results/lease release from a stale owner.

30. Does emergency stop undo everything? It requests controller abort, waits/forces recorded cleanup and sweeps recognized stopped processes/helpers/activation/scopes. It does not recover host failure, automatically restart every missing unrecorded component, or restore lost intent details. Inspect its report and remaining health.

31. Can SQLite quick_check prove no production was lost? No. It checks DB structure. Nondecreasing IDs are another limited check. A killed transaction can roll back without corruption and still lose a business operation. Application ground-truth scoring offers different accounting evidence.

32. Can an API outage leave tag ingestion healthy? Yes, OPC agent may write while API is down. API-derived metric probes become unavailable, so the graph cannot directly prove an ingestion halt. Keep direct observer/database evidence separate from API access.

33. What if first compound step fails recovery? Current loop stops further injection; final verification still runs. Aggregate logic may not explicitly reject incomplete planned execution. A new plan result must mark remaining steps skipped and require all steps for complete success.

34. Why not run two submit calls concurrently for multiple injection? They race a non-locked lease, lose coordinated journals and have no overlap/conflict/cleanup schedule. Sequential steps under one owner are the existing path; deliberate overlap needs new design and policy.

35. Can valid citations prove an answer is true? No. They prove IDs belong to supplied context. Claim entailment, omitted qualifiers and wrong interpretation remain model-quality issues. Use labelled evaluation and evidence validation; no automatic remediation based on citation syntax.

36. Why can four backend workers consume much more RAM? Each process independently loads its embedding/reranking models; on-disk lock doesn't share model objects. One worker reduces memory at throughput cost. Warm/download model separately for production and respect 120-second worker timeout limits.

37. Why does health work while plant discovery fails in the cloud sandbox? HTTP reaches services regardless of reading process environment, while adapter identity requires `MES_PLANT_NAME` from psutil environ. AccessDenied causes discovery to skip processes. Normal process-access execution passed here; changing recovery to ignore missing identity would weaken safety.

38. What are host/container boundary mistakes? Container localhost is not host/another service; internal DNS differs; native psutil cannot safely represent container identities; killing one replica may not cause outage; SQLite checks don't apply to PostgreSQL. A container adapter must define exact blast radius/restoration/evidence.

39. How would you build recommendations safely? Deterministic evidence package + optional approved runbook retrieval + read-only recommender + typed outputs/evidence references + separate review. Policy-validate any next experiment and require explicit execution approval. No model shell/policy-edit/production action tool.

40. How would you demonstrate API-crash success? Show compatible baseline, approved bounded crash, process exit/observed request failure, original-settings restart, stable authenticated/manufacturing recovery and consistency evidence, then report data uncertainty. Native crash exists; rich timing/new deployment adapters are future work.

41. Which claims should you refuse to make? Official Day 2 completion without requirements; production readiness; all tests/all stacks verified; no data lost from quick_check; simultaneous injections implemented; recommendation engine complete; actual ML detection; Docker run just because Compose exists; fault success from submit acceptance or a PID.

42. Which trade-off is most central? The project chooses a narrow, inspectable local experiment environment and honest evidence over broad uncontrolled fault reach. That supports useful development/testing, while current coordination/crash-window issues limit how strong its safety claims can be. Explain both the strengths and practical limits with file references.

14.4 Glossary

Term	Meaning in this project
MES	Manufacturing execution system that observes/books plant work and quality
OPC UA	Machine telemetry protocol; replay server publishes, agent subscribes
Replay / seed	Recorded deterministic simulated signals / reproducibility input
Infrastructure fault	Real disturbance to software process/resource availability/performance
Application fault	Scripted manufacturing/observer event in ephemeral scenario
Baseline / post	Observation phases before injection / after verified recovery
Blast radius / allowlist	Resources a fault can affect / symbolic approved target inventory
Policy / preflight	Static acceptance/bounds / runtime readiness and fault-specific checks
Intent / undo record	Persisted plan for effect / information required to restore it
Lease / fencing	Active ownership record / rejecting actions from stale owners (proposed stronger enforcement)
Watchdog	Separate local process that cleans dead/expired controller records
Cleanup / removal / recovery	Restore action / proof disturbance ended / proof application works again
Monotonic clock	Time that does not jump with wall-clock adjustments; durations/expiry use it
SIGKILL / SIGTERM	Forced / cooperative process termination
SIGSTOP / SIGCONT	Suspend / resume process execution
RSS	Resident process memory; not total cgroup working set
cgroup / scope	Kernel resource group / delegated systemd process grouping
MemoryHigh / MemoryMax	Reclaim pressure threshold / hard cap that can OOM-kill
CPU quota / pressure	Kernel time-budget cap / contention from another scheduled workload
Prometheus / Grafana	Metric collection/query store / visualization layer
Counter / gauge	Accumulating quantity / current value; exporter mostly gauges
Histogram / p95	Bucketed distribution / percentile; SQL bucket estimate is an upper bound
OEE / coverage	Availability×performance×quality / how much behavior was observed
Ground truth / booking band	Simulator's known output / acceptable accounting range from harness
MCP / tool call	Structured model-client tool protocol / named schema-bound invocation
ASGI / worker	Async Python web interface / serving process with its own model memory
RAG / embedding	Retrieval-supported answer generation / numeric semantic representation
Cosine / reranking	Normalized vector similarity / secondary query-passage ordering
Citation / entailment	Source identifier in answer / whether source supports the actual claim
FAISS / generation / CURRENT	Vector index library / immutable snapshot / atomic snapshot pointer
fsync / atomic rename	Flush file data / publish complete pathname replacement
Docker DNS / replica	Internal service-name resolution / another serving API instance
Recovery with data findings	Core behavior restored but data evidence fails; distinct from full recovery

14.5 Final cross-check and unresolved gaps

This guide was checked against the newly created audit and source execution traces, current README/docs/configuration/definitions, and targeted implementation reads across controller, faults, policy, agents, MCP, adapters, persistence, monitoring, backend RAG and upstream Docker. All 14 requested topic areas are represented. Implemented future-topic pieces are separated from proposed additions. Verified tests/baseline are separated from unrun integrations.

Unresolved source gaps: original Bosch problem statement and Day 2 requirements; missing referenced experiment/resource reports; missing external OPC injection checkout; unspecified agreed recommendation/overlap/deployment designs. Unresolved implementation risks are indexed in audit and §11. No claim of official requirement verification, production safety, complete AI recommendation, complete compound guarantees or all-stack validation is made. When new requirement documents arrive, map their exact terminology to source symbols, test evidence and remaining work before updating completion claims.